SALARY PREDICTION WEB APPLICATION

Complete Beginner Learning & Deployment Guide

AWS S3 + IAM + EC2 + Python + Flask + Linear Regression + Gunicorn + Nginx + systemd

Purpose

This self-contained guide explains the complete project from zero: private CSV storage in S3, EC2 permissions through IAM, Python data loading, Linear Regression, Flask, Gunicorn, systemd, Nginx, testing, troubleshooting, security and cleanup.

1. Project Overview

The application predicts salary from years of experience. The CSV is stored privately in S3. Ubuntu EC2 runs the application. Flask handles requests, scikit-learn provides Linear Regression, Gunicorn runs Flask, systemd manages the service, and Nginx is the public HTTP front door.

Component	Job	Why
S3	Store CSV	Private durable object storage
IAM role	Authorize EC2	No hard-coded AWS keys
EC2	Run application	Cloud Linux server
pandas	Read data	CSV/data processing
scikit-learn	Train model	Linear Regression
Flask	Web application	Browser request handling
Gunicorn	WSGI server	Production-style app serving
systemd	Service manager	Start/restart at boot
Nginx	Reverse proxy	Public HTTP entry point

2. Architecture

ARCHITECTURE

```
Browser
   |
   | HTTP :80
   v
Nginx
   |
   | Unix socket
   v
Gunicorn
   |
   v
Flask (app.py)
```

```
   |
   | boto3 + IAM role
   v
Private S3 CSV
   |
   v
pandas -> LinearRegression -> prediction
   |
   v
HTML response -> Browser
```

Mental model: S3 is the storage room, IAM is the permission desk, EC2 is the computer, Flask is the application, Gunicorn is the application server, and Nginx is the public front door.

3. Prerequisites

- AWS account with permission to create S3, IAM and EC2 resources.
- Windows PC with PowerShell/Windows Terminal.
- An SSH .pem key pair.
- salary_prediction_500.csv.
- Awareness that AWS resources may incur charges.

4. Dataset and ML Concept

Column	Role	Example
YearsExperience	Feature / X	0.6
Salary	Target / y	257000

Linear Regression learns a relationship between YearsExperience and Salary.

ML CORE

```
X = df[["YearsExperience"]]
y = df["Salary"]

model = LinearRegression()
model.fit(X, y)

prediction = model.predict([[5]])
```

5. Create the Private S3 Bucket

1. Open AWS Console → S3 → Create bucket.
2. Enter a globally unique bucket name such as salary-prediction-yourname-2026.

3. Choose a region. Keep EC2 and S3 in the same region for this lab.
4. Leave Object Ownership as Bucket owner enforced.
5. Keep Block all public access enabled.
6. Versioning is optional for this lab; it is useful when you want older object versions.
7. Keep server-side encryption enabled.
8. Choose Create bucket.

Security
The CSV does not need to be public. EC2 will read it using AWS authentication.

6. Upload the CSV

9. Open the bucket.
10. Choose Upload → Add files.
11. Select salary_prediction_500.csv.
12. Choose Upload.
13. Confirm the object key is exactly salary_prediction_500.csv.

S3 object names are case-sensitive. The name in Python must match the uploaded object.

7. Create the EC2 IAM Role

14. IAM → Roles → Create role.
15. Trusted entity: AWS service.
16. Use case: EC2.
17. For this learning lab attach AmazonS3ReadOnlyAccess.
18. Name it SalaryPredictionEC2Role.
19. Create the role.

For production, use least privilege rather than broad S3 read access.

OPTIONAL LEAST-PRIVILEGE POLICY

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::YOUR_BUCKET_NAME/salary_prediction_500.csv"
  }]
}
```

Never hard-code AWS keys
An EC2 IAM role lets boto3 obtain temporary credentials. Do not put access keys or secret keys

inside app.py.

8. Launch Ubuntu EC2

Option	Beginner choice	Explanation
Name	salary-prediction-server	Easy identification
AMI	Ubuntu Server LTS	Stable Linux OS
Architecture	x86_64	Simple compatibility choice
Instance type	Small eligible type such as t3.micro	Enough for this small lab; verify current pricing/eligibility
Key pair	Your .pem key	SSH authentication
VPC/Subnet	Default is fine	Simplifies networking
Public IP	Enabled	Allows SSH/browser access
Storage	Default small EBS volume	Application is small
IAM instance profile	SalaryPredictionEC2Role	Gives EC2 S3 permission

Do not make port 5000 public. The final application is exposed through Nginx on port 80.

9. Security Group

Rule	Port	Source	Purpose
SSH	22	My IP	Admin access
HTTP	80	0.0.0.0/0	Public website
HTTPS	443	0.0.0.0/0	Only after HTTPS is configured
Flask	5000	None publicly	Keep behind Nginx

10. SSH from Windows

POWERSHELL

```
cd "C:\path\to\your\key"
ssh -i "your-key.pem" ubuntu@YOUR_PUBLIC_IP
```

Replace the path, key name and public IPv4 address with your own values.

11. Linux Basics

Command	Meaning
pwd	Show current directory
ls -la	List files, including hidden files
cd	Change directory
mkdir	Create directory
nano	Edit text file
sudo	Run with administrator privileges

cat	Display a file
clear	Clear terminal
exit	End SSH session

12. Install Python and Create venv

```
TERMINAL

python3 --version
sudo apt update
sudo apt install -y python3-pip python3-venv
mkdir -p /home/ubuntu/salary-prediction
cd /home/ubuntu/salary-prediction
python3 -m venv venv
source venv/bin/activate
python --version
pip --version
```

A virtual environment isolates this application's Python packages from the system Python installation.

13. Create Project Files

```
TERMINAL

cd /home/ubuntu/salary-prediction
mkdir -p templates static
touch app.py requirements.txt test_s3.py test_csv.py
ls -la
```

14. Install Dependencies

```
TERMINAL

pip install flask pandas scikit-learn boto3 gunicorn s3fs
pip freeze > requirements.txt
```

Package	Purpose
Flask	Web framework
pandas	Data/CSV processing
scikit-learn	Machine learning
boto3	AWS SDK
gunicorn	WSGI application server
s3fs	S3 filesystem integration
fsspec	Filesystem abstraction used by S3 integration

15. Test S3 Access

test_s3.py

```
import boto3

s3 = boto3.client("s3")

response = s3.list_buckets()

for bucket in response["Buckets"]:
    print(bucket["Name"])
```

TERMINAL

```
python test_s3.py
```

A successful result proves the Python environment can authenticate to AWS through the EC2 role.

16. Test CSV Reading

test_csv.py

```
import pandas as pd

bucket_name = "YOUR_BUCKET_NAME"
file_name = "salary_prediction_500.csv"

s3_path = f"s3://{bucket_name}/{file_name}"

df = pd.read_csv(s3_path)

print(df.head())
print()
print("Number of rows:", len(df))
```

TERMINAL

```
python test_csv.py
```

The original lab returned 500 rows. That verifies the S3 object can be read as a pandas DataFrame.

17. The fsspec Error

The project initially produced an error similar to ModuleNotFoundError: No module named 'fsspec'. When pandas reads an s3:// URL, its filesystem integration relies on packages such as s3fs and fsspec.

FIX

```
pip install fsspec
pip install s3fs
python test_csv.py
```

This is a common Python lesson: installing one library may install or require supporting libraries.

18. Build app.py

app.py

```
from flask import Flask, render_template, request
import pandas as pd
from sklearn.linear_model import LinearRegression
import boto3
from io import BytesIO

app = Flask(__name__)

BUCKET_NAME = "YOUR_BUCKET_NAME"
FILE_NAME = "salary_prediction_500.csv"

def load_data_from_s3():
    s3 = boto3.client("s3")
    response = s3.get_object(Bucket=BUCKET_NAME, Key=FILE_NAME)
    data = response["Body"].read()
    return pd.read_csv(BytesIO(data))

def train_model():
    df = load_data_from_s3()
    X = df[["YearsExperience"]]
    y = df["Salary"]

    model = LinearRegression()
    model.fit(X, y)
    return model

model = train_model()

@app.route("/", methods=["GET", "POST"])
def home():
    prediction = None
    error = None
```

```
    if request.method == "POST":
        try:
            years = float(request.form["years"])

            if years < 0:
                raise ValueError

            prediction = model.predict([[years]])[0]

        except ValueError:
            error = "Please enter a valid number."

    return render_template(
        "index.html",
        prediction=prediction,
        error=error
    )

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Part	Explanation
Flask imports	Create the web application
boto3	Access S3
load_data_from_s3()	Download the private CSV
BytesIO	Treat downloaded bytes as a file
train_model()	Prepare X/y and train Linear Regression
model = train_model()	Train when application starts
@app.route	Defines the website route
request.form	Reads form input
model.predict	Calculates predicted salary
render_template	Sends values to HTML
app.run	Development server only

19. Build index.html

templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Salary Prediction</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <main class="container">
        <section class="card">
            <h1>Salary Prediction</h1>
```

```
            <p>Enter your years of experience to estimate salary.</p>

            <form method="POST">
                <label for="years">Years of Experience</label>
                <input type="number" id="years" name="years"
                       step="0.1" min="0" required>
                <button type="submit">Predict Salary</button>
            </form>

            {% if prediction is not none %}
                <div class="result">
                    Predicted Salary:
                    ₹{{ "{:,.2f}".format(prediction) }}
                </div>
            {% endif %}

            {% if error %}
                <div class="error">{{ error }}</div>
            {% endif %}

            <p class="info">
                Model: Linear Regression<br>
                Dataset: salary_prediction_500.csv
            </p>
        </section>
    </main>
</body>
</html>
```

Flask uses Jinja syntax such as {{ prediction }} and {% if ... %} to insert Python values and conditionally display content.

20. Build style.css

static/style.css

```
body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #f3f6fb;
}

.container {
    min-height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
    padding: 20px;
}

.card {
```

```
    width: 100%;
    max-width: 480px;
    background: white;
    padding: 30px;
    border-radius: 16px;
    box-shadow: 0 8px 25px rgba(0,0,0,0.08);
}

input, button {
    width: 100%;
    box-sizing: border-box;
    padding: 12px;
    margin-top: 8px;
    margin-bottom: 16px;
    border-radius: 8px;
    border: 1px solid #ccc;
}

button {
    background: #2563eb;
    color: white;
    border: none;
}

.result {
    margin-top: 20px;
    padding: 15px;
    background: #e8f5e9;
    border-radius: 8px;
    font-weight: bold;
}

.error {
    margin-top: 20px;
    padding: 15px;
    background: #ffebee;
    border-radius: 8px;
    color: #b71c1c;
}
```

21. Final Directory Structure

DIRECTORY

```
salary-prediction/
├── app.py
├── requirements.txt
├── test_s3.py
├── test_csv.py
├── templates/
│   └── index.html
├── static/
```

```
│   └── style.css
└── venv/
```

22. Test Flask

TERMINAL

```
cd /home/ubuntu/salary-prediction
source venv/bin/activate
python app.py
```

Flask should listen on 0.0.0.0:5000. Test locally from EC2:

TEST

```
curl http://127.0.0.1:5000/
```

Stop it with Ctrl+C. Do not treat the development server as the final production service.

23. Why Gunicorn?

Gunicorn is a WSGI application server. Flask is the application framework; Gunicorn is the server process that runs the Flask application in a production-style setup.

TERMINAL

```
pip install gunicorn
gunicorn --workers 2 --bind 127.0.0.1:5000 app:app
```

In app:app, the first app means app.py and the second app means the Flask object named app.

24. systemd Service

systemd starts the application automatically, keeps it running, restarts it after failure, and provides logs/status.

/etc/systemd/system/salary-prediction.service

```
[Unit]
Description=Salary Prediction Flask Application
After=network.target

[Service]
```

```
User=ubuntu
Group=www-data
WorkingDirectory=/home/ubuntu/salary-prediction
Environment="PATH=/home/ubuntu/salary-prediction/venv/bin"
RuntimeDirectory=salary-prediction
RuntimeDirectoryMode=0755
ExecStart=/home/ubuntu/salary-prediction/venv/bin/gunicorn \
    --workers 2 \
    --bind unix:/run/salary-prediction/salary-prediction.sock \
    app:app
Restart=always

[Install]
WantedBy=multi-user.target
```

TERMINAL

```
sudo systemctl daemon-reload
sudo systemctl enable salary-prediction
sudo systemctl restart salary-prediction
sudo systemctl status salary-prediction
```

Section	Meaning
[Unit]	Service identity/order
[Service]	How the program runs
[Install]	Boot-time enablement

DIAGNOSTICS

```
sudo journalctl -u salary-prediction -n 100 --no-pager
ls -la /run/salary-prediction/
```

25. Nginx Reverse Proxy

Nginx accepts public HTTP traffic and forwards it to Gunicorn through the Unix socket.

/etc/nginx/sites-available/salary-prediction

```
server {
    listen 80;
    server_name _;

    location / {
        proxy_pass http://unix:/run/salary-prediction/salary-prediction.sock;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
```

```
    }
}
```

TERMINAL

```
sudo ln -s /etc/nginx/sites-available/salary-prediction /etc/nginx/sites-enabled/salary-prediction
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl restart nginx
sudo systemctl enable nginx
sudo systemctl status nginx
```

26. 502 Bad Gateway: What Happened in This Project

The browser initially returned 502. Nginx's error log showed Permission denied while connecting to the Unix socket. This proved that Flask itself was not necessarily the problem.

Check	Purpose
sudo systemctl status salary-prediction	Is Gunicorn running?
sudo journalctl -u salary-prediction -n 100 --no-pager	Why did Gunicorn/systemd fail?
sudo tail -n 100 /var/log/nginx/error.log	Why can Nginx not reach upstream?
ls -la /run/salary-prediction/	Does the socket exist and who owns it?
curl --unix-socket /run/salary-prediction/salary-prediction.sock http://localhost/	Does Flask/Gunicorn work directly?

Key lesson

A healthy Nginx process can still return 502 if the upstream socket is missing or inaccessible. Check the upstream path, socket permissions and directory traversal permissions.

27. Final Browser Test

BROWSER

```
http://YOUR_EC2_PUBLIC_IPV4
```

Enter a value such as 5 and choose Predict Salary. The browser request travels through Nginx → Gunicorn → Flask → model and returns HTML.

28. End-to-End Request Flow

20. Browser sends HTTP POST to port 80.

21. Nginx receives it.
22. Nginx forwards it through the Unix socket.
23. Gunicorn passes it to Flask.
24. Flask reads the form value.
25. Linear Regression predicts salary.
26. Flask renders index.html.
27. Gunicorn returns the response to Nginx.
28. Nginx sends the response to the browser.

At startup, train_model() also causes boto3 to read the private S3 object using the EC2 IAM role, pandas to build the DataFrame, and scikit-learn to train the model.

29. Troubleshooting

Symptom	Likely cause	First checks
SSH timeout	Port 22/IP/network	Security Group, public IP, instance state
Website timeout	Port 80/Nginx	Security Group, nginx status
502	Upstream failure	Nginx error log, Gunicorn status
Socket Permission denied	Nginx cannot access socket/path	owner/group/mode and directory permissions
Socket missing	Gunicorn failed	journalctl service logs
S3 AccessDenied	IAM issue	EC2 role and policy
NoSuchKey	Wrong object name	Bucket/key spelling and case
fsspec missing	Dependency issue	pip install fsspec/s3fs
Gunicorn inactive	Application/config crash	journalctl -u salary-prediction
nginx -t fails	Syntax/config problem	Read exact Nginx error

30. Security Best Practices

- Keep S3 private.
- Use IAM roles, not hard-coded access keys.
- Use least privilege in production.
- Restrict SSH to My IP where practical.
- Do not expose port 5000.
- Use HTTPS/TLS for real production use.
- Never commit .pem files or credentials.
- Keep OS, packages and Nginx updated.
- Use monitoring/logging for production.

31. Cost and Cleanup

29. Terminate EC2 when finished if it is no longer needed.
30. Delete the S3 object and bucket if no longer needed.
31. Delete the lab IAM role if unused.
32. Check for unused public IPv4/Elastic IP resources.

33. Review AWS Cost Management.

Stopping an EC2 instance does not necessarily remove all charges; attached storage and other resources may still incur costs.

32. AWS Component Summary

Service	Simple mental model	Project role
S3	Storage room	CSV
IAM	Permission desk	EC2 authorization
EC2	Cloud computer	Runs app
VPC	Network	Connectivity
Subnet	Network section	EC2 placement
Security Group	Firewall	Inbound traffic
EBS	Hard disk	OS/project files

33. Command Reference

Command	Meaning
pwd	Where am I?
ls -la	What is here?
cd	Move
mkdir	Create folder
nano	Edit
sudo	Administrator
python3	Python
pip	Python packages
source venv/bin/activate	Activate venv
systemctl	Manage service
journalctl	Read service logs
nginx -t	Validate Nginx
curl	Test HTTP

34. Package Reference

Package	Meaning
Flask	Web framework
pandas	Data processing
scikit-learn	ML
boto3	AWS SDK
s3fs	S3 filesystem
fsspec	Filesystem abstraction
gunicorn	WSGI server

35. Learning Checkpoints

Checkpoint A — AWS

- Why should S3 remain private?
- Why does EC2 need an IAM role?
- Why is port 5000 not public?

Checkpoint B — Linux

- Create and enter a directory.
- Create a file with touch.
- Read a file with cat.

Checkpoint C — ML

- What are X and y?
- What does fit() do?
- What does predict() do?

Checkpoint D — Deployment

- Why Gunicorn?
- Why Nginx?
- What is a Unix socket?
- Why systemd?

Mini Exercises

34. Change the web page title.
35. Add a maximum experience validation.
36. Try three Gunicorn workers.
37. Replace broad S3 read permission with a least-privilege policy.
38. Add CloudWatch CPU monitoring and an SNS alarm.
39. For production, add HTTPS and a domain.

36. Final Mental Model

Remember five layers

DATA → MODEL → APPLICATION → SERVER → NETWORK

DATA: S3 stores the CSV.
MODEL: scikit-learn learns the relationship.
APPLICATION: Flask handles requests.
SERVER: Gunicorn + systemd keep Python running.
NETWORK: Nginx + Security Group expose the app.

When something fails, first identify which layer failed. Then inspect that layer's logs, permissions

and configuration.